Summary of Modules

In the following table the purposes of relevant Python modules are described, which were used for Master Thesis "On the Influence of Temporal Sequence Length and Label Granularity in Machine Learning-Based ECG Classification", Leuphana University, 2026

Notes:

1. Input data files and output data files are stored in module specific paths. The original used paths are still contained in the code and must be adapted to the actual used paths.
2. Input or output datafile descriptions like "***Input:*** *X_np_arrays_x.npz, Y_data_x.pkl, x= 1, 2 or 3*" refer to different files, resulting from different samplings of X- and y-data by Phyton code parameter "random_state".

#	Modul	Description	Input- / output data
1	fetch_ECG_raw_data. ipynb	The module reads the raw data and saves it as Numpy arrays or as a dataframe.	**Input**: path = 'D:/ptb-xl-a-large-publicly-available-electrocardiography-dataset-1.0.3/' **Output:** **spc_statements.pkl** = dataframe with all spc-statements and its internal correlations **X_float32.npy**= file of numpy array with ECG data (resp. values) for each of the 12 leads (shape: 21199, 1000, 12) **Y_data.pkl** = pickl data with dataframe of all ECG information (e. g. label) for each ECG
2	shifting_ECGs. ipynb	This module shifts the start of the ECG data for augmentation purposes. The new places (= shifted place) of the data are calculated according to the formula *new place = old place + shift value (mod 1000)*	**Input:** X_float32.npy or X_mixed.npz.npy

		That is, if a new place has a place number > 999 the counting is continued at the beginning of the sequence. The shift value is drawn randomly from a list [100, 150, 200, ..., 900, 950], that is each ECG will be randomly shifted according to the shift value drawn from the list. All 12 leads of a certain ECG are shifted about the same shift value. The shifting of the ECG values are performed K times (K is 2 by default). Care is taken to ensure that repeated shifting is not done with two identical shift values. The resulting two datasets (K=2) with shifted ECG values (2 times 21199 values) are concatenated to the original dataset. The module also allows for choosing different lengths of the ECG sequence. For shortened ECG length (e.g. to 800 datapoints) the list of shift values is adjusted accordingly [100, 150, 200, ..., 700, 750] Further the X-dataset with mixed length, which results from module ESG_mixed_1.ipynb can be read and shifted as described. The dataset with the mixed ECG lengths will be treated the same way as the ECGs with length of 100 datapoints.	**Output:** X_augmented.npz X_augmented_accomp.pkl X_mixed_augmented.npz
3	ECG_mixed.ipynb	This module reads the original ECG data and shortens the ECG sequence randomly to 800, 600, 400, 200 datapoints or let the number of datapoints unchanged at 1000. If a sequence is shortened the missing values are replenished with 0.00 values to gain effective 1000 datapoint again.	**Input:** X_float32.npy **Output:** X_mixed.npz

		Resulting data are saved in the folder D:/ECG/X_mixed	
4	fetch_handwriting_data.ipynb	Extracts the handwriting data from the BRUSH dataset: y-data: the text snippets that are written by different writers. x-data: the digitalized x- and y-coordinates of the digitalized handwritten text snippets. Only the y-coordinates are further processed and used. The data are divided into different classes (= a certain text snippet) with a certain number of instances (=writers). The data are divided into training-, validation and test-data. Processing x-data (=y-coordinates) of writers: • The sequences are of different lengths and therefore the data are extended to unique 1000 datapoints by stacking the datapoint onto each other until 1000 datapoints are reached. • Rescaling the y-coordinates into a range similar to the range of the ECG data • Each x-data (=y-coordinates of text snippet of one writer) is replicated to get entire 12 sequence. This is necessary to align the X-data matrix of all handwritten text to the shape of the ECG X-data matrix for combining these two types of data matrixes in module " combine_hndwrtg_and_ECG.ipynb"	**Input:** "D:/BRUSH/refined_BRUSH/BRUSH/130/2_resample25" **Output:** extracted_data_dict.pkl (all extracted data, not processed, reload this data for repeated processing) X_hwrtng_np_arrays_x.npz (X-data; training, validation, test) x=1, 2 or 3 Y_hwrtng_data_x.pkl (y-data, training, validation, test) x=1, 2 or 3
5	ECG_like_handwriting. ipynb	With this module the original X-data of the ECGs are read, and two different leads of a certain ECG is chosen for graphical display of the data.	**Input:** X_float32.npy

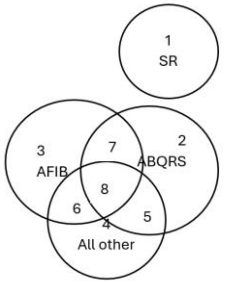
		The purpose is to show the similarity of this kind of ECG data representation with handwriting.	**Output: -**
	label_scenario_**1**. ipynb to label_scenario_**5**. ipynb	The purpose of these five modules is the reordering and division of the original y-data (which is a data frame with several information for each of the ECG) to build different groups/classes according to the label scenarios described in the master thesis document. The different y-label scenarios can be built for the original X-data as well as for the X-data which contains two-fold shifted X-data (see module shiftig_ESG_2_modified.ipynb). The modified y-data extended with additional created labels (according to the label scenario) are split into training-, validation- and test sets. After loading the X-data (or the shifted X-data) the X-data are assigned to the corresponding training-, validation-, and test sets of the y-data. All data (training-, validation- and test-data for y-data and X-data) are saved in module specific folders.	
6	label_scenario_1. ipynb	According to label scenario 1 this module splits the Y-data into two groups and labels the Y-data with additional labels: 1. Sinus rhythm (=normal) EGC with label "SR" 2. All other ECG with deviation from the normal ECG with additional label "all_other"	**Input:** 'Y_data.pkl' scp_statements.csv X_augmented_accomp.pkl X_mixed_augmented.npz.npy X_float32.npy **Output:** X_np_arrays_x.npz Y_data_x.pkl

			x= 1, 2 or 3
7	label_scenario_2. ipynb	According to label scenario 2 this module splits the Y-data into four groups and labels the Y-data with additional labels: 1. Sinus rhythm (=normal) EGC with label "SR" 2. All Y-data with one single label, and which build groups with not less than 49 instances. These Y-data are combined to one group and labeled with additional label "single_label" 3. All Y-data with two labels, and which build groups with not less than 49 instances. These Y-data are combined to one group and labeled with additional label "binary_label" 4. All Y-data which do not belong to group one, two or three are combined to a fourth group and labeled with additional label "all_other"	**Input:** Y_data.pkl scp_statements.csv X_float32.npy X_augmented.npz.npy **Output:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3
8	label_scenario_3. ipynb	According to label scenario 3 this module splits the Y-data into four groups and labels the Y-data with additional labels: 1. Sinus rhythm (=normal) EGC with label "SR" 2. All ECG which are labeled with one single "form" label. These ECG are combined to one group and assigned with label "form"	**Input:** Y_data.pkl scp_statements.csv X_float32.npy X_augmented_accomp.pkl X_augmented.npz.npy **Output:** X_np_arrays_x.npz

		3. All ECG which are labeled with one single "rhythm" label. These ECG are combined to one group and assigned with label "rhythm" 4. All ECGs which do not belong to group one, two, or three are combined into a fourth group and labeled with the label "all_other"	Y_data_x.pkl x= 1, 2 or 3
9	label_scenario_4. ipynb	According to label scenario 4 this module selects from Y-data all instances with a single label which builds groups with not less than 49 instances. These criteria fulfill the following groups: 'SR', 'ABQRS', 'NDT', 'AFIB', 'SARRH', 'SBRAD', 'VCLVH', 'STACH', 'PVC', 'PACE', 'NST_', 'QWAVE', 'STD_', 'LPR', 'PAC', 'NT_', 'LVOLT', 'SVARR', 'LNGQT',	**Input:** Y_data.pkl scp_statements.csv X_float32.npy X_augmented_accomp.pkl X_augmented.npz.npy **Output:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3

		'INVT' The class labeled with "all_other" contained all the instances of the y-data which are not contained in the 20 single label y-data above.	
10	label_scenario_5. ipynb	According to label scenario 5 this module splits the Y-data into eight groups and labels the Y-data with additional labels: There are 4 groups with single labels: 1. Sinus rhythm (=normal) EGC with label "SR" 2. ABQRS 3. AFIB 4. All other alone (contain no y-data instance with SR, or ABQRS, or AFIB) 3 groups are built by binary intersections of two of the previous groups: 5. ABQRS + all other 6. AFIB + all other 7. ABQRS + AFIB The last group is an intersection of three of the single labeled groups: 8. ABQRS + AFIB + all other For the Y-data in the SR group, no intersections with other labels exist.	**Input:** Y_data.pkl scp_statements.csv X_float32.npy X_augmented_accomp.pkl X_augmented.npz.npy **Output:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3

<table>
<tr><td></td><td></td><td>The following table displays the multi-hot codification for a multi label classification (group 5, 6, 7, 8 contains multi labels)

<table><tr><td>1</td><td>SR</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>2</td><td>ABQRS</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>3</td><td>AFIB</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>4</td><td>All other</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>5</td><td>ABQRS + all other</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>6</td><td>AFIB + all other</td><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>7</td><td>ABQRS + AFIB</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>8</td><td>ABQRS + AFIB + all other</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table> 
</td><td></td></tr>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td>11</td><td>prepare_data_for_training_1.ipynb</td><td>This module takes the X and y data which are prepared from label_scenario_1.Ipynb and stored in the folder XY_diff_class_0
and reduces the sequence length of the X-data, transforms the codes_2 columns of the y-data df into integers (1 = SR and 0 = all_others), transforms the X-data into numpy arrays of the appropriate dimension and stores the complete X- and y-data (X_train, y_train, X_val, y_val, X_test, y_test) in a numpy npz-file</td><td>Input:
X_np_arrays_x.npz
Y_data_x.pkl

x= 1, 2 or 3

Output:
X_y_input.npz</td></tr>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td>12</td><td>prepare_data_for_training_2.ipynb</td><td>This module takes the X and y data which are prepared</td><td>Input:</td></tr>
</table>

		a) from label_scenario_2. ipynb and stored in the folder XY_diff_class_**1** and transforms the codes_2 (resp. codes) columns of the y-data df into integers (0 = SR and 1 = binary, 2=single, 3=all_other) , b) from label_scenario_3. Ipynb and stored in the folder XY_diff_class_**2** and transforms the codes_2 (resp. codes) columns of the y-data df into integers (0 = SR and 1 = form, 2=rhythm, 3=all_other), c) from label_scenario_4. ipynb and stored in the folder XY_diff_class_**3** and transforms the codes_2 (resp. codes) columns of the y-data df into integers (0 = SR and 1 = label 2, 2= label 3, …, 20=all_other) Transforms the X- data into numpy arrays of the appropriate dimension and stores the complete X- and y-data (X_train, y_train, X_val, y_val, X_test, y_test) in a numpy npz-file	X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3 **Output:** X_y_input.npz
13	prepare_data_for_training_3.ipynb	This module prepares data for multilabel classification.	**Input:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3 **Output:** X_y_input.npz

14	prepare_data_for_training_4.ipynb	This module prepares data for pre-training with handwritten text snippets.	**Input:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3 **Output:** X_y_input.npz
15	combine_hndwrtg_and_ECG.ipynb	This module prepares combines X-data and y-data of ECG and handwritten text for X/y-train X/y-val X/y-test	**Input:** X_np_arrays_x.npz X_hwrtng_np_arrays_x.npz Y_data_x.pkl Y_hwrtng_data_x.pkl x= 1, 2 or 3 **Output:** X_np_arrays_x.npz Y_data_x.pkl x= 1, 2 or 3
16	min_max_of_X_data.ipynb	This module extracts the min. and max values of the ECG values from all 21k ECGs (ECG lead can be chosen) and calculates the mean value.	**Input:** X_float32.npy **Output:** -
17	CNN_final_model_2.ipynb	This is the module for training and prediction for binary classification ("SR", "all_other") with CNN model	**Input:** X_y_input.npz **Output:**

			best_binary_cnn.keras, final_binary_cnn.keras, final_metrics.csv, final_metrics.json, model_architecture.json, model_configuration.json, model_summary.txt, test_classification_report.csv, test_confusion_matrix.csv, test_predictions.csv, training_classification_report.csv, training_confusion_matrix.csv, training_history.csv, training_log.csv, training_predictions.csv, validation_classification_report.csv, validation_confusion_matrix.csv, validation_predictions.csv
18	GRU_final_model_2.ipynb	Training and prediction like CNN_final_model_2.ipynb but with GRU model	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
19	CNN_final_model_3.ipynb	This module based on CNN_final_model_2.ipynb adapted to classification of four classes.	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
20	GRU_final_model_3.ipynb	Like CNN_final_model_3.ipynb but with GRU model	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
21	CNN_final_model_4.ipynb	This module based on CNN_final_model_2.ipynb adapted to classification of 21-classes.	**Input:** X_y_input.npz

		Note: hyperparameter "learning rate" reduced and "batch size" increased for more stability Based on	**Output:** according to CNN_final_model_2.ipynb
22	GRU_final_model_4.ipynb	Like CNN_final_model_4.ipynb but with GRU model	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
23	CNN_final_model_5.ipynb	This model based on CNN_final_model_2.ipynb adapted to 13-classes classification (label scenario 5) Note: hyperparameter "learning rate" reduced and "batch size" increased for more stability	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
24	CNN_final_model_6.ipynb	This model uses the pretrained weights of the module " CNN_final_model_5.ipynb" and used these weights as starting parameters for the training of binary classification SR vs. all_other.	**Input:** X_y_input.npz best_13_class_cnn.keras **Output:** according to CNN_final_model_2.ipynb
25	CNN_final_model_7. ipynb	CNN model adapted to multi label classification	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb
26	GRU_final_model_7. ipynb	Like CNN_final_model_7. ipynb but with GRU model	**Input:** X_y_input.npz **Output:** according to CNN_final_model_2.ipynb